

Detección de Fraude en Transacciones Financieras usando Aprendizaje Automático

Edwin Iñiguez Moncada¹[A01637064], Hiber Leonardo Macías
Cancino¹[A01782765], Álvaro Solano González¹[A01643948], and Carlos
Francisco Sánchez Llanes¹[A01741201]

Tecnológico de Monterrey

Abstract. En este reporte se describe el desarrollo de una solución basada en ciencia de datos para la detección de fraude en transacciones financieras. Se abordan las etapas de limpieza y transformación de datos, selección y entrenamiento de modelos, evaluación, refinamiento y la implementación de una interfaz web para el uso del modelo. Se presentan experimentos, resultados y conclusiones, así como la contribución de cada miembro del equipo.

1 Introducción

El fraude en transacciones financieras representa uno de los principales retos para instituciones bancarias y empresas de tecnología financiera en la actualidad. El crecimiento del comercio electrónico y la digitalización de los servicios financieros han incrementado tanto el volumen de transacciones como la sofisticación de los métodos de fraude, lo que hace indispensable el desarrollo de sistemas automáticos de detección.

El objetivo de este proyecto es diseñar, implementar y evaluar una solución basada en técnicas de ciencia de datos y aprendizaje automático para la detección de fraude en transacciones financieras. Para ello, se trabajó con un conjunto de datos realista y público, proveniente de Kaggle [7], que contiene información relevante sobre transacciones, clientes y dispositivos, permitiendo abordar el problema desde una perspectiva integral.

El desarrollo del proyecto abarca todas las etapas del ciclo de vida de un proyecto de ciencia de datos: desde la limpieza y transformación de los datos, la selección y entrenamiento de modelos, la evaluación y refinamiento de los mismos, hasta la implementación de una interfaz web que permite utilizar el modelo entrenado en un entorno práctico.

Este reporte documenta el proceso seguido, las decisiones tomadas, los experimentos realizados y los resultados obtenidos, con el fin de proporcionar una visión clara y reproducible de la solución propuesta para la detección de fraude.

2 Metodología

El desarrollo de la solución para la detección de fraude siguió el ciclo de vida típico de un proyecto de ciencia de datos, estructurándose en las siguientes etapas:

2.1 1. Análisis exploratorio y limpieza de datos

Se inició con una exploración exhaustiva del conjunto de datos `Base_clean.parquet`, el cual es una versión procesada del dataset original *Bank Account Fraud Dataset (NeurIPS 2022)* disponible en Kaggle [7]. Este dataset contiene información de 32 variables relacionadas con transacciones bancarias, perfiles de clientes y dispositivos utilizados. Durante la fase exploratoria se analizaron las distribuciones de las variables, se identificaron valores centinela (-1) que indicaban información desconocida, y se calcularon matrices de correlación para entender las relaciones entre variables. Se crearon visualizaciones específicas para analizar patrones en casos de fraude versus casos legítimos.

2.2 2. Transformación y preprocesamiento

Se desarrolló un pipeline robusto de preprocesamiento que incluye: (1) tratamiento de valores centinela mediante imputación con mediana para variables continuas, (2) creación de variables indicadoras binarias para valores desconocidos, (3) detección y eliminación de outliers utilizando el método del rango intercuartílico (IQR), (4) normalización de variables numéricas con `StandardScaler`, y (5) codificación de variables categóricas mediante `OneHotEncoder`. Todo el preprocesamiento se integró en un pipeline de `scikit-learn` para garantizar reproducibilidad y prevenir fugas de información entre conjuntos de entrenamiento y prueba.

2.3 3. Selección y entrenamiento de modelos

Se implementaron y compararon múltiples algoritmos de clasificación supervisada: regresión logística con regularización L1 y L2, máquinas de soporte vectorial (SVM) con kernels RBF y lineal, redes neuronales multicapa (MLPClassifier), árboles de decisión y random forest. Para abordar el severo desbalance de clases (aproximadamente 94% de casos no fraudulentos), se integró la técnica de sobremuestreo sintético SMOTE en el pipeline de entrenamiento. La evaluación se realizó utilizando validación cruzada estratificada para mantener la proporción de clases.

2.4 4. Evaluación y selección de métricas

Dado el contexto del problema, se priorizaron métricas que reflejan el desempeño en clases desbalanceadas: F1-score, precisión y recall. Se implementó validación

cruzada estratificada con 5 particiones para obtener estimaciones robustas del desempeño. Además, se generaron curvas de aprendizaje para evaluar la capacidad de generalización de los modelos y detectar posibles problemas de sobreajuste o subajuste.

2.5 5. Análisis de interpretabilidad

Se analizaron los coeficientes de la regresión logística y las importancias de características en modelos basados en árboles para identificar las variables más relevantes en la predicción de fraude. Esta interpretabilidad es crucial para la validación del dominio y la confianza en las predicciones del modelo.

2.6 6. Implementación de la solución

Se desarrolló una arquitectura de dos componentes: (1) un backend API en Flask que carga el modelo entrenado y expone un endpoint REST para predicciones, y (2) una interfaz web desarrollada en Next.js que permite a los usuarios ingresar datos de transacciones y obtener predicciones en tiempo real. La solución incluye validación de entrada y manejo de errores para garantizar robustez en producción.

Esta metodología permitió abordar el problema de manera estructurada, asegurando la calidad de los datos, la robustez del modelo y la facilidad de uso de la solución final.

3 Experimentos

Para evaluar la efectividad de los modelos propuestos, se diseñó una serie de experimentos sistemáticos utilizando múltiples conjuntos de datos derivados del dataset base. A continuación se describen los principales experimentos realizados:

3.1 1. Análisis exploratorio por variantes del dataset

Se realizó un análisis exploratorio de datos (EDA) independiente para cada variante del dataset (Base, Variant I, II, III, IV, V), documentado en notebooks específicos (EDA.ipynb, EDA_1.ipynb hasta EDA_5.ipynb). Este análisis reveló diferencias en las distribuciones de variables entre variantes, patrones específicos de valores atípicos, y variaciones en la proporción de casos fraudulentos. Los hallazgos permitieron adaptar las estrategias de preprocesamiento para cada variante.

3.2 2. Comparación exhaustiva de algoritmos

Se implementaron y evaluaron sistemáticamente múltiples algoritmos de clasificación:

- Regresión logística con regularización L1, L2 y Elastic Net (documentado en `regressionLogistica.ipynb`)
- Máquinas de Soporte Vectorial con kernels RBF y lineal (`SVM_MLP_BASE.ipynb`, `svc_base.ipynb`)
- Redes neuronales multicapa con diferentes arquitecturas (`MLP_BASE.ipynb`)
- Modelos basados en árboles incluyendo Random Forest (`Arboles_ensambles/`)

Cada modelo fue integrado en pipelines automatizados que incluían el pre-procesamiento específico y técnicas de balanceamiento de clases.

3.3 3. Estrategias para datos desbalanceados

Dado que el dataset presenta un severo desbalance (94% casos no fraudulentos), se experimentó con múltiples técnicas:

- Sobremuestreo sintético usando SMOTE
- Pesos de clase balanceados en algoritmos que lo soportan
- Submuestreo de la clase mayoritaria
- Combinaciones de técnicas de balanceamiento

3.4 4. Validación cruzada estratificada

Se implementó un esquema robusto de validación utilizando `StratifiedKFold` con 5 particiones, garantizando que la proporción de clases se mantenga constante en cada fold. Las métricas de evaluación incluyeron F1-score, precisión, recall, AUC-ROC y matrices de confusión detalladas.

3.5 5. Análisis de curvas de aprendizaje

Se generaron curvas de aprendizaje para los modelos más prometedores, evaluando el comportamiento del F1-score en función del tamaño del conjunto de entrenamiento. Este análisis permitió identificar si los modelos se beneficiarían de más datos o si presentaban problemas de sobreajuste.

3.6 6. Preprocesamiento unificado de múltiples datasets

Se desarrolló un pipeline automatizado (`preprocess_all_datasets.py`) capaz de procesar simultáneamente todas las variantes del dataset, aplicando transformaciones consistentes y generando un conjunto unificado para entrenamiento. Esto permitió aprovechar la información de todas las variantes para mejorar la robustez del modelo final.

4 Resultados

En esta sección se presentan los resultados obtenidos a partir de los experimentos realizados con los diferentes modelos y configuraciones desarrolladas en el proyecto.

4.1 1. Análisis exploratorio de datos

El análisis exploratorio reveló características importantes del dataset:

- El dataset contiene 1,048,575 transacciones con 32 variables predictoras
- Desbalance severo de clases: aproximadamente 94% casos no fraudulentos vs 6% fraudulentos
- Presencia de valores centinela (-1) en 6 variables clave: prev_address_months_count, current_address_months_count, credit_risk_score, bank_months_count, session_length_in_minutes, y device_distinct_emails_8w
- Identificación de variables categóricas críticas: payment_type, employment_status, housing_status, source, device_os

4.2 2. Efectividad del preprocesamiento

Las técnicas de limpieza y transformación implementadas demostraron ser efectivas:

- La creación de variables indicadoras para valores desconocidos (prev_address_unknown, bank_months_unknown) proporcionó información adicional valiosa
- El recorte de outliers usando IQR mejoró la estabilidad de los modelos
- La normalización StandardScaler resultó crucial para algoritmos sensibles a escala como SVM y redes neuronales
- OneHotEncoder para variables categóricas permitió una representación adecuada para los algoritmos

4.3 3. Comparación de modelos

Los resultados de la comparación de algoritmos mostraron diferencias significativas en el desempeño:

- **Regresión Logística:** Se destacó por su interpretabilidad y balance entre precisión y recall, logrando un desempeño consistente con regularización L1
- **SVM:** Mostró ser una implementación demasiado pesada con kernel RBF, con demasiado costo computacional (incapaz de terminar el entrenamiento despues de 12 horas).
- **Random Forest:** Demostró robustez, especialmente útil para análisis de importancia de variables.
- **MLPClassifier:** Presentó potencial alto pero requirió ajuste cuidadoso de hiperparámetros para evitar sobreajuste.

4.4 4. Impacto de SMOTE en datos desbalanceados

La aplicación de SMOTE demostró ser fundamental para mejorar el desempeño:

- Incremento significativo en el recall (de 0.41 a 0.81 en regresion logistica) para la clase minoritaria (fraude)
- Mejora en el F1-score general manteniendo la precisión estable, relativa a la anterior.
- Reducción de falsos negativos en el contexto de detección de fraude, un poco.

4.5 5. Variables más importantes

El análisis de interpretabilidad reveló las variables más influyentes en la predicción de fraude:

- **credit_risk_score:** Principal predictor con fuerte correlación negativa con fraude
- **income:** Ingresos bajos asociados con mayor riesgo de fraude
- **prev_address_months_count:** Historial de direcciones corto indica mayor riesgo
- **session_length_in_minutes:** Sesiones anormalmente cortas o largas correlacionan con fraude
- **employment_status y housing_status:** Estados inestables aumentan probabilidad de fraude

4.6 6. Implementación web funcional

Se logró desarrollar una demo local de una aplicación web que incluye:

- Backend Flask con API REST para predicciones en tiempo real
- Frontend Next.js con interfaz intuitiva para ingreso de datos
- Pipeline de preprocesamiento integrado que replica exactamente las transformaciones del entrenamiento
- Validación de entrada y manejo de errores
- Funcionalidad de entrada por lotes para pruebas rápidas

4.7 7. Rendimiento del sistema

La solución implementada demostró:

- Tiempo de respuesta inferior a 500ms por predicción individual
- Capacidad de procesar múltiples transacciones simultáneamente
- Estabilidad en el preprocesamiento sin fugas de información
- Compatibilidad con diferentes formatos de entrada de datos

5 Conclusiones

El desarrollo de este proyecto permitió abordar de manera integral el problema de la detección de fraude en transacciones bancarias, aplicando técnicas de ciencia de datos y aprendizaje automático en un contexto realista con datos altamente desbalanceados. A través de un proceso sistemático que abarcó desde el análisis exploratorio hasta la implementación de una solución web funcional, se logró construir un sistema con mucha capacidad para mejorar.

Entre los principales aprendizajes y logros destacan:

- **Criticidad del análisis exploratorio:** La exploración detallada de múltiples variantes del dataset reveló patrones específicos de fraude y guió decisiones cruciales de preprocesamiento. El análisis de correlaciones, distribuciones y valores atípicos fue fundamental para el éxito del proyecto.
- **Importancia del preprocesamiento inteligente:** La gestión cuidadosa de valores centinela, la creación de variables indicadoras para información desconocida, y el tratamiento específico de outliers demostraron ser más importantes que la sofisticación del algoritmo de clasificación.
- **Efectividad de técnicas de balanceamiento:** SMOTE demostró ser crucial para abordar el desbalance severo de clases, mejorando significativamente la capacidad del modelo para detectar fraudes sin comprometer excesivamente la precisión.
- **Valor de la interpretabilidad:** Los modelos interpretables como regresión logística no solo proporcionaron buen desempeño, sino que permitieron validar las predicciones contra el conocimiento del dominio financiero.
- **Importancia de la validación robusta:** La validación cruzada estratificada y el análisis de curvas de aprendizaje fueron fundamentales para asegurar la generalización y detectar problemas de sobreajuste.
- **Viabilidad de implementación práctica:** La integración exitosa del modelo en una API Flask y una interfaz web Next.js demostró que es posible llevar modelos de ciencia de datos a aplicaciones de producción utilizando herramientas modernas.

Oportunidades de mejora

Para incrementar el desempeño de futuros modelos de detección de fraude, se recomienda:

- **Análisis temporal más profundo:** Incorporar análisis de series de tiempo para detectar patrones temporales de fraude y comportamientos estacionales.
- **Ingeniería de características avanzada:** Crear variables derivadas basadas en conocimiento del dominio, como ratios entre variables financieras, indicadores de comportamiento atípico, y métricas de consistencia entre campos relacionados.
- **Técnicas de detección de anomalías:** Complementar los modelos supervisados con métodos no supervisados como Isolation Forest o autoencoders para detectar patrones de fraude previamente desconocidos.
- **Validación externa y datos sintéticos:** Utilizar técnicas de generación de datos sintéticos para aumentar la diversidad del conjunto de entrenamiento y validar el modelo con datasets externos.
- **Monitoreo continuo:** Implementar sistemas de monitoreo de deriva de datos (data drift) para detectar cuándo el modelo necesita reentrenamiento debido a cambios en los patrones de fraude.

En conclusión, la solución propuesta demuestra que es posible detectar fraudes de manera efectiva utilizando herramientas modernas de ciencia de datos, pero el éxito depende críticamente de la calidad del análisis de datos, la comprensión del dominio del problema, y la aplicación rigurosa de técnicas de validación.

Conclusiones personales (Edwin)

Este proyecto representó una experiencia integral en ciencia de datos aplicada que me permitió desarrollar habilidades técnicas y metodológicas cruciales. El trabajo con un problema real de detección de fraude me enseñó que el éxito en ciencia de datos depende más de la calidad del análisis y preprocesamiento de datos que de la sofisticación del algoritmo elegido.

La exploración detallada de múltiples variantes del dataset me ayudó a desarrollar intuición sobre cómo los datos pueden revelar patrones ocultos cuando se analizan correctamente. Aprendí que cada decisión en el preprocesamiento, desde el tratamiento de valores atípicos hasta la codificación de variables categóricas, tiene un impacto directo y medible en el desempeño final del modelo.

La implementación de pipelines automatizados y reproducibles me enseñó la importancia de la ingeniería de software en ciencia de datos. La experiencia de integrar modelos entrenados en aplicaciones web funcionales me mostró cómo cerrar la brecha entre la investigación y la aplicación práctica, un aspecto crucial pero a menudo subestimado en proyectos académicos.

Trabajar con datos altamente desbalanceados me forzó a reconsiderar métricas tradicionales de evaluación y a priorizar el contexto del negocio sobre la precisión bruta. Esta experiencia será invaluable en futuros proyectos donde las consideraciones prácticas y éticas son tan importantes como el desempeño técnico.

6 Bibliografía

References

1. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
2. Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning. *Journal of Machine Learning Research*, 18(17), 1-5.
3. Springer. (2023). *Springer Lecture Notes in Computer Science (LNCS) - Instructions for Authors*. Disponible en: <https://www.springer.com/gp/computer-science/lncs/conference-proceedings-guidelines>
4. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321-357.
5. Vercel. (2024). Next.js Documentation. Disponible en: <https://nextjs.org/docs>
6. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.
7. S. G. P. Jesus. Bank Account Fraud Dataset (NeurIPS 2022). Kaggle. Disponible en: <https://www.kaggle.com/datasets/sgpjesus/bank-account-fraud-dataset-neurips-2022>

Contribución de los integrantes

A continuación se describe la contribución de cada integrante del equipo al desarrollo de la solución:

- **Edwin:** Responsable del análisis exploratorio exhaustivo de datos (EDA.ipynb, EDA_1.ipynb a EDA_5.ipynb), desarrollo e implementación del modelo de regresión logística (regressionLogistica.ipynb, regresion_logistica.py), diseño y construcción del pipeline de preprocesamiento automatizado (preprocess_all_datasets.py, preprocess_RegressionLogistica.ipynb), transformación y limpieza de múltiples variantes del dataset, desarrollo de la API Flask y la interfaz web Next.js para el despliegue del modelo, y redacción completa del reporte técnico.
- **Carlos:** Contribuyó con experimentación en modelos de red neuronal (MLP-Classifer) documentada en notebooks específicos.
- **Hiber Leonardo:** Trabajó en la implementación de modelos basados en árboles de decisión y random forest en la carpeta Arboles_ensambles.
- **Alvaro:** No realizó contribuciones sustanciales al proyecto. Anunció su intención de participar en el análisis de datos y creación del reporte durante los últimos días antes de la entrega, pero no completó ninguna de estas tareas.